AArch64 Memory & Cache System — Technical Reference Workbook 1
Chapter 9 — Full System Initialization Sequence: MMU, Caches, SMP, DMA, and Instruction/Data Visibility (Based on ARM64 CortexA72) 1
9.1 Scope 1
9.2 Initialization Overview 1
9.3 StepbyStep Initialization Sequence 2
Step 0 — Foundational Cache Terminology (Prerequisite) 2
Step 1 — Disable MMU and Caches 2
Step 2 — Configure MAIR_EL1 (Memory Attributes) 2
Step 3 — Configure TCR_EL1 (Translation Control) 3
Step 4 — Configure TTBR0_EL1 / TTBR1_EL1 (Translation Tables) 3
Step 5 — Optional ImplementationSpecific Configuration 3
Step 6 — Enable MMU 4
Step 7 — Enable Caches 4
Step 8 — Instruction Visibility Preparation 4
Step 9 — Data Visibility Preparation 5
Step 10 — Device Memory Configuration 5
Step 11 — SMP BringUp 6
CPU0 Responsibilities 6
CPU1 Responsibilities 6
Step 12 — DMA Subsystem Initialization 6
CPU → DMA (Device Reads CPU Data) 7
DMA → CPU (Device Writes Data) 7
9.4 Unified Initialization Flowchart 7
9.5 Exercises (Technical Reference) 8
Exercise 9.1 — Initialization Ordering 8
Exercise 9.2 — Cache Enablement 8
Exercise 9.3 — SMP BringUp 8
Exercise 9.4 — DMA Visibility 8

Exercise 9.5 — MMIO Safety..... 8

AArch64 Memory & Cache System — Technical Reference Workbook

Chapter 9 — Full System Initialization Sequence: MMU, Caches, SMP, DMA, and Instruction/Data Visibility (Based on ARM64 Cortex-A72)

9.1 Scope

This chapter defines the **complete, end-to-end initialization sequence** required to safely bring up an ARM64 Cortex-A72 system. It integrates:

- Memory attribute configuration
- MMU initialization
- Cache enablement
- Instruction visibility rules
- Data visibility rules
- SMP coherency requirements
- DMA interaction rules
- Device memory ordering rules

This chapter serves as the **master reference sequence** for all system software.

9.2 Initialization Overview

The full initialization sequence consists of:

1. **Foundational configuration**
2. **Memory attribute system setup**
3. **MMU activation**
4. **Cache activation**

5. **Instruction visibility preparation**
6. **Data visibility preparation**
7. **Device memory configuration**
8. **SMP bring-up**
9. **DMA subsystem initialization**

Each step must be executed in the correct order to ensure architectural correctness.

9.3 Step-by-Step Initialization Sequence

Step 0 — Foundational Cache Terminology (Prerequisite)

Before initialization, the following concepts must be understood:

- Cache line (64 bytes)
- Set, way, associativity
- Tag RAM vs data RAM
- Dirty vs clean lines
- Write-back vs write-through
- PoU (L2)
- PoC (system-wide domain)
- FEQ (Fill/Evict Queue)
- Snoop logic
- Register slices

(See Chapter 0 for full definitions.)

Step 1 — Disable MMU and Caches

Required to ensure deterministic initialization.

```
SCTLR_EL1.M = 0
SCTLR_EL1.C = 0
SCTLR_EL1.I = 0
```

Caches must be off before configuring attributes.

Step 2 — Configure MAIR_EL1 (Memory Attributes)

Define attribute encodings:

- Normal, Write-Back, Read/Write-Allocate → `0xFF`
- Normal, Non-Cacheable → `0x44`
- Device-nGnRnE → `0x00`

MAIR must be configured **before** TCR/TTBR.

Step 3 — Configure TCR_EL1 (Translation Control)

Configure:

- Shareability (Inner-shareable for SMP)
- Cacheability (WB WA for Normal memory)
- Granule size (typically 4 KB)
- Physical address size
- TTBR0/TTBR1 split

Incorrect TCR configuration breaks coherency.

Step 4 — Configure TTBR0_EL1 / TTBR1_EL1 (Translation Tables)

Install translation tables with correct attributes:

- Normal memory → cacheable
- MMIO → Device-nGnRnE
- DMA buffers → Normal cacheable or Non-cacheable (depending on design)
- Kernel text → Normal cacheable
- Bootloader scratch → Normal cacheable

TTBR configuration determines visibility semantics.

Step 5 — Optional Implementation-Specific Configuration

Examples:

- L1D prefetcher tuning
- Branch prediction restrictions
- ACTLR_EL1 fields
- Vendor-specific coherency controls

These do not affect architectural semantics.

Step 6 — Enable MMU

```
SCTLR_EL1.M = 1
```

Effects:

- Activates attribute system
- Activates program-flow prediction
- Makes PoU/PoC meaningful
- Enables correct DC/IC semantics

Step 7 — Enable Caches

```
SCTLR_EL1.C = 1
SCTLR_EL1.I = 1
```

Effects:

- L1D and L1I become active
- Dirty lines may exist
- Instruction caching begins
- Cache maintenance becomes required

Step 8 — Instruction Visibility Preparation

Required when:

- Bootloader patches code
- Kernel installs trampolines
- Hypervisor modifies guest code
- CPU0 prepares code for CPU1

Sequence:

```
DC   CVAU, xN
IC   IVAU, xN
DSB  ISH
ISB
```

Ensures updated instructions propagate to PoU and L1I.

Step 9 — Data Visibility Preparation

Required when:

- CPU prepares data for DMA
- CPU prepares data for other cores
- CPU prepares data for other clusters

Sequence:

```
DC   CVAC, xN
DSB  ISH
```

Ensures dirty lines propagate to PoC.

Step 10 — Device Memory Configuration

MMIO must be marked **Device-nGnRnE**.

Rules:

- No caching
- No DC/IC operations
- No speculation
- No prefetch
- Ordering enforced only by barriers

Typical access patterns:

```
STR wX, [MMIO]
DSB SY
```

Step 11 — SMP Bring-Up

CPU0 Responsibilities

Prepare bootstrap code:

```
DC   CVAU, bootstrap
IC   IVAU, bootstrap
DSB  ISH
ISB
```

Release secondary cores.

CPU1 Responsibilities

```
ISB
```

Ensures instruction refetch from PoU.

Step 12 — DMA Subsystem Initialization

CPU → DMA (Device Reads CPU Data)

```
DC   CVAC, xN
DSB  ISH
```

DMA → CPU (Device Writes Data)

```
DC   CIVAC, xN
DSB  ISH
```

DMA interacts only with PoC.

9.4 Unified Initialization Flowchart

```
Disable MMU/Caches
      ↓
Configure MAIR
      ↓
Configure TCR
      ↓
Install TTBR tables
      ↓
Optional ACTLR/Prefetcher config
      ↓
Enable MMU
      ↓
Enable Caches
      ↓
Instruction visibility prep (CVAU + IVAU + DSB + ISB)
      ↓
Data visibility prep (CVAC + DSB)
      ↓
Configure MMIO (Device-nGnRnE)
      ↓
```

```
SMP bring-up (CPU0: CVAU+IVAU+DSB+ISB, CPU1: ISB)
      ↓
DMA subsystem initialization (CVAC/CIVAC + DSB)
```

9.5 Exercises (Technical Reference)

Exercise 9.1 — Initialization Ordering

Explain why MAIR must be configured before enabling the MMU.

Exercise 9.2 — Cache Enablement

Describe the architectural consequences of enabling SCTLR_EL1.C before TTBR tables are installed.

Exercise 9.3 — SMP Bring-Up

Explain why CPU1 must execute ISB during secondary core initialization.

Exercise 9.4 — DMA Visibility

Describe why DC CIVAC is required after DMA writes to memory.

Exercise 9.5 — MMIO Safety

Explain why DC/IC operations must not be used on Device-nGnRnE regions.